TRANSFORMERS · A WORKED EXAMPLE · ~12 MIN READ

The Next Word

How a transformer turns a sentence into a prediction — every step, every number.

> *No prior knowledge assumed. We pick one short sentence and follow it through all five stages of a transformer. Every matrix, dot product, and probability below is real and checks out — you can verify it with a calculator.*

A transformer does exactly one thing: **given some words, predict the next word.** Everything it does is in service of that. So let's give it a sentence and watch it predict.

`"the cat sat on the"` → `?`

By the end you'll have computed, by hand, why the answer comes out **"mat"**. There are five stages:

1. **Embeddings + position** — turn each word, and where it sits, into numbers
2. **Attention** — let words gather context from each other ★ the heart
3. **MLP** — let each word digest what it gathered
4. **Layers** — stack 2+3 many times
5. **Prediction** — turn the result into the next word

Stage 1 · Embeddings: words become numbers

A computer can't compute on the letters `c-a-t`. So every word is assigned a short list of numbers — a **vector**. Words used similarly get similar vectors, so *meaning becomes geometry*: nearby vectors = related words.

> **Why.** Numbers are the only thing we can do math on. A vector places each word as a point in space, and "closeness" in that space can encode meaning. The model *learns* these numbers during training; here we just pick simple ones.

COMPUTE · OUR 5 TOKENS AS 4-DIM VECTORS

the	1	0	0	1
cat	0	2	1	0
sat	1	0	2	0
on	0	1	0	1
the	1	0	0	1

Real models use ~4000 numbers per word; we use 4 so we can see them. Note the two `"the"`s have identical *content* here — in a moment, position will pull them apart.

Formally, the content embeddings live in a learned matrix E with one **row** per vocabulary word. Looking up token t_i selects its row (we treat every token vector as a row throughout — the convention papers and code use):

$$e_i = E_{t_i,:}, \qquad E \in \mathbb{R}^{|V| \times d} \tag{1}$$

where d is the model dimension (4 in our toy example, ~4000 in real models) and $|V|$ is the vocabulary size. This e_i captures *what* the word means — but not yet *where* it sits.

Where each word sits: positional encoding

Here's a catch we must fix *before* attention. The attention step (next) treats the words as an unordered *set* — on its own it cannot tell `"the cat sat"` from `"sat cat the."` But order *is* meaning ("dog bites man" ≠ "man bites dog"). So we stamp each word with **where it sits** by adding a position vector p_i to its content embedding:

$$x_i = e_i + p_i \tag{2}$$

Each slot gets its own distinct p_i, so the same word in two places ends up with two different input vectors x_i. (Real models build p_i from smooth functions — sinusoids, or rotations like RoPE; we use simple stamps so you can add them by hand.)

COMPUTE · STAMP POSITION ONTO CONTENT: X = E + P

`position p (one stamp per slot)`

the @0	0	0	0	0
cat @1	1	0	0	0
sat @2	0	1	0	0
on @3	0	0	1	0
the @4	0	0	0	1

`input x = e + p (what attention actually sees)`

the @0	1	0	0	1
cat @1	1	2	1	0
sat @2	1	1	2	0
on @3	0	1	1	1
the @4	1	0	0	2

The two `"the"`s were identical as content, but now differ: `[1,0,0,1]` vs `[1,0,0,2]`. Position pulled them apart — *before* attention even runs.

Checkpoint. Each word is now an input vector $x_i = e_i + p_i$ — *what* it means plus *where* it sits. The two `"the"`s already differ. ✓

Stage 2 · Attention ★ the heart

Here's the problem attention exists to solve. To predict what follows `"the"`, the model must know *what the sentence is about* — but the final `"the"`'s vector knows nothing about "cat" or "sat" on its own. **Attention lets it look at the other words and pull in their meaning.**

> **The intuition.** Each word broadcasts a question — *"who here is relevant to me?"* — compares it against what every word advertises, and then mixes in the content of whichever words matched. We'll do this for the last word, `"the"`, since that's the one predicting the next token.

Query, Key, Value — three roles per word

Every word produces three vectors by multiplying its input vector x with three **learned** matrices. Think of searching YouTube: your **query** (what you type) is matched against each video's **key** (its title), and you receive its **value** (the video itself).

$$q = x\,W_Q \qquad k = x\,W_K \qquad v = x\,W_V \tag{3}$$

Each weight matrix has shape $d \times d_k$ (here 4×3), turning a 4-dim word vector into a 3-dim query, key, or value.

COMPUTE · THE QUERY FOR THE LAST WORD "THE", X = [1,0,0,2]

W_Q (4×3)

1	0	1
0	1	1
1	0	0
0	1	0

$q = [1,0,0,2] \cdot W_Q$, e.g. $q_2 = 1\cdot0+0\cdot1+0\cdot0+2\cdot1 = 2$

x dotted with each column → q = 1 2 1

Doing the same with W_K and W_V for *every* word gives a key and a value for each. (Same arithmetic, just repeated — we'll show the results.)

Step A — how relevant is each word? (dot products)

We score relevance by the **dot product** of our query with each word's key — one number measuring alignment. Bigger = more relevant.

$$\text{score}_i = q \cdot k_i = \sum_j q_j \, k_{i,j} \tag{4}$$

COMPUTE · Q·K FOR EACH WORD (Q = [1,2,1])

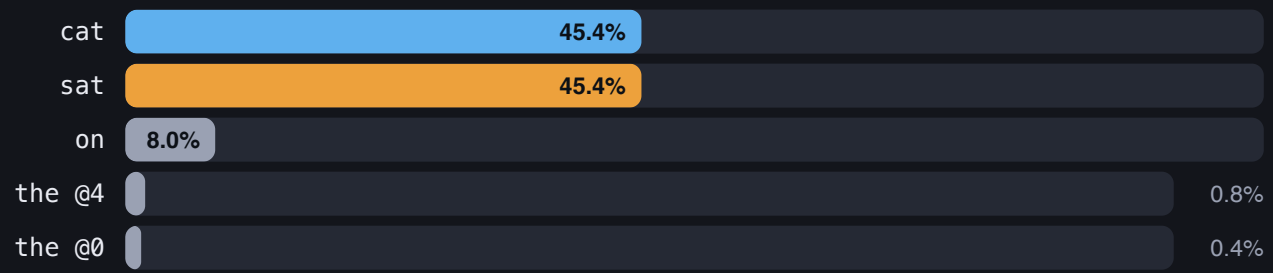
word	key k	q · k
the @0	1 0 1	2
cat @1	3 3 1	10
sat @2	2 3 2	10
on @3	1 2 2	7
the @4	1 0 2	3

e.g. cat: $1{\cdot}3 + 2{\cdot}3 + 1{\cdot}1 = 10$. "cat" and "sat" *tie* for highest — the subject and the verb.

Step B — turn scores into percentages (softmax)

Raw scores aren't percentages. **Softmax** turns any list of numbers into positive fractions that sum to 100% — bigger scores get exponentially bigger shares. (We first divide by $\sqrt{d_k} = \sqrt{3}$, a standard step that keeps scores from blowing up.)

$$\text{weight}_i = \frac{e^{\,\text{score}_i/\sqrt{d_k}}}{\sum_j e^{\,\text{score}_j/\sqrt{d_k}}} \tag{5}$$



The model splits its attention almost equally between **"cat" (the subject) and "sat" (the verb)** — the two most informative words for guessing what comes next. That's exactly the context you'd want.

Causal masking. Because the goal is to predict the *next* word, a real model **masks** attention so each position may attend only to itself and *earlier* words — never future ones (otherwise it could just peek at the answer). We sidestep this by computing only the *final* token, which is already allowed to see every word before it — so no masking is needed for our example.

Step C — mix the values by those weights

Now build the new vector for `"the"` as a weighted blend of every word's **value**, using the percentages above. This blended vector is "the" *with context baked in*.

$$\text{output} = \sum_i \text{weight}_i \, v_i \tag{6}$$

COMPUTE · THE CONTEXT-AWARE OUTPUT VECTOR

values v (one row per word)

the @0	3	0	3
cat @1	2	4	1
sat @2	2	5	1
on @3	1	3	2
the @4	4	0	5

$0.454 \cdot v_{cat} + 0.454 \cdot v_{sat} + \ldots \rightarrow$ output = **1.94** **4.32** **1.12**

The big middle value (4.32) comes straight from cat & sat, whose values are large in that slot. "the" has absorbed the subject-and-verb meaning.

The famous one-line formula just packs Steps A–C — Eqs. (4), (5), (6) — into one expression, and now you can read it:

$$\text{Attention}(Q,K,V) = \underbrace{\text{softmax}\!\left(\frac{QK^\top}{\sqrt{d_k}}\right)}_{\text{the weights (A,B)}} V \quad \underbrace{}_{\text{the blend (C)}} \tag{7}$$

Multi-head attention. A word often needs several kinds of context at once (subject? tense? tone?). So this whole operation runs H times in parallel — each "head" h with its own W_Q^h, W_K^h, W_V^h — and the results are concatenated and mixed by an output matrix W_O:

$$\text{head}_h = \text{Attention}\big(XW_Q^h,\ XW_K^h,\ XW_V^h\big) \tag{8}$$

$$\text{MultiHead}(X) = \big[\text{head}_1 \,\|\, \text{head}_2 \,\|\, \cdots \,\|\, \text{head}_H\big]\, W_O \tag{9}$$

where $\|$ means “stack side by side.” Same machinery as above, just repeated and recombined.

Toy-model caveat. W_O maps the concatenated heads back to the model dimension d, so every layer’s output matches its input size — that’s what makes the residual sum in Eq. (11) valid. For visibility our toy uses a *single* head with $d_k = 3$ and omits W_O, so its attention output stays 3-dim ($\neq d{=}4$). That’s why we feed that output straight into prediction rather than running it through the residual and MLP on actual numbers — those stay conceptual here.

Checkpoint. Attention = score relevance (q·k) → softmax to weights → blend the values. The final `"the"` is now context-aware: mostly cat + sat. ✓

Stage 3 · MLP: each word digests privately

Attention was *social* — words traded information. Now each word, on its own, processes what it gathered. The output vector is passed through a small two-step network: **expand** to a larger size (room to compute combinations), apply a nonlinearity, then **squeeze** back down.

In symbols, it's two matrix multiplies with a nonlinearity ϕ in between — applied to each word's vector x independently:

$$\mathrm{MLP}(x) = \phi\big(x\,W_1 + b_1\big)\,W_2 + b_2 \tag{10}$$

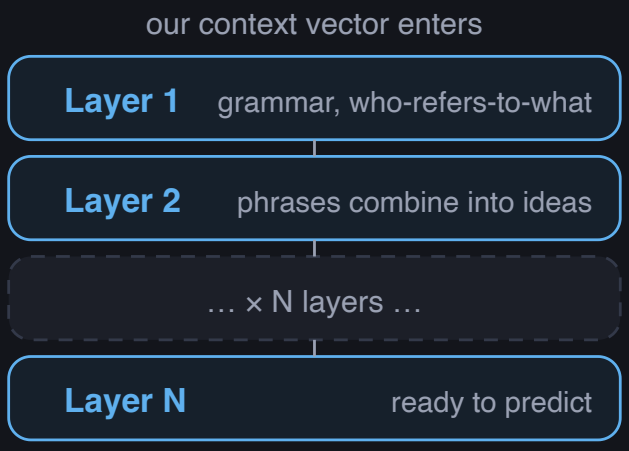
W_1 projects up to a larger hidden size (the "room to think"), ϕ is a nonlinearity (e.g. *ReLU* or *GELU*) that lets the network compute non-linear combinations, and W_2 projects back down. Without ϕ, the two matrices would collapse into one and the layer could only do linear maps.

> **The intuition.** Attention is the meeting where everyone shares; the MLP is each person back at their desk, digesting what was said. No word talks to another here — it's pure per-word refinement. **Attention + MLP together = one "layer."**

Checkpoint. The MLP refines each word's vector alone. One round of (attention + MLP) = one layer. ✓

Stage 4 · Layers: stack it deep

One layer gives shallow understanding. So we **repeat the same (attention + MLP) block many times** — dozens, in real models. Each layer's output is the next layer's input, so understanding compounds: early layers catch grammar, later layers assemble abstract meaning.



One layer is two sub-steps, and each *adds* its result back to its input — a **residual connection** — so information is never thrown away:

$$x' = x + \mathrm{MultiHead}(x) \qquad y = x' + \mathrm{MLP}(x') \tag{11}$$

Stacking N such layers just feeds each one's output into the next, starting from the embeddings $x^{(0)}$:

$$x^{(\ell+1)} = \mathrm{Layer}_\ell\big(x^{(\ell)}\big), \qquad \ell = 0, 1, \ldots, N-1 \tag{12}$$

The residual form $x_{\text{out}} = x_{\text{in}} + \mathrm{sublayer}(x_{\text{in}})$ is what lets you stack dozens of layers without the signal degrading: each layer only contributes a refinement on top of what's already there. (Real models also insert a normalization step inside each sub-layer — omitted here as plumbing, not concept.)

Checkpoint. Depth = repeated gather-and-digest. Shallow patterns early, deep meaning late. ✓

Stage 5 · Prediction: numbers back into a word

After the last layer, we take the vector at the **final position** (it now carries the whole sentence's context) and score every word in the vocabulary. Each candidate word has its own **output vector**; the dot product with our context vector is its **logit** (raw score). Softmax — the same one as Eq. (5) — turns logits into probabilities.

Concretely, an **unembedding** matrix W_U (one **column** per vocabulary word) turns the final vector $x_{\text{last}}^{(N)}$ into a **logit** per word; softmax turns those into probabilities:

$$z = x_{\text{last}}^{(N)} W_U, \qquad W_U \in \mathbb{R}^{d \times |V|}, \quad z \in \mathbb{R}^{|V|} \tag{13}$$

$$P(\text{next} = w) = \frac{e^{z_w}}{\sum_{w'} e^{z_{w'}}}, \qquad \widehat{w} = \arg\max_w P(\text{next} = w) \tag{14}$$

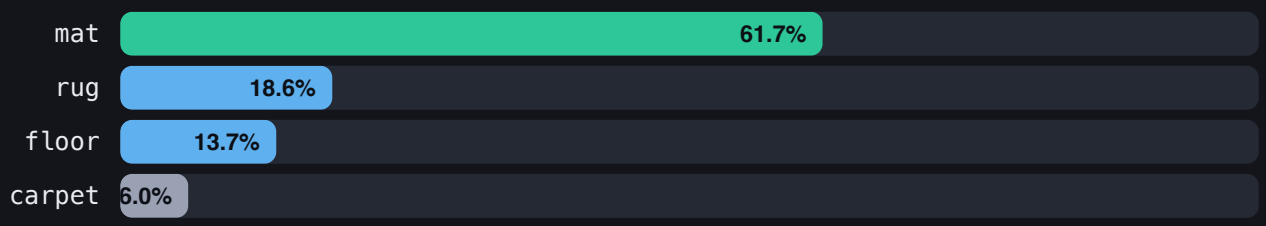
Each **column** of W_U is a word's "signature"; the dot product z_w measures how well the context vector matches it. We then pick the highest-probability word (or *sample* from the distribution).

> **The intuition.** This is attention's trick reused: match our context vector against each candidate word's "signature." The best-aligned word wins. We'll use our attention output as the context vector to keep the thread concrete.

COMPUTE · SCORE 4 CANDIDATE NEXT-WORDS (CONTEXT = [1.94, 4.32, 1.12])

word	W_U column	logit (dot)
mat	2 −2 −1	**−5.88**
rug	−2 −1 1	**−7.08**
floor	−1 −1 −1	**−7.38**
carpet	−2 −1 0	**−8.20**

e.g. mat: $2(1.94) - 2(4.32) - 1(1.12) = -5.88$. Logits are usually negative — only their *differences* matter. After softmax over the vocabulary:

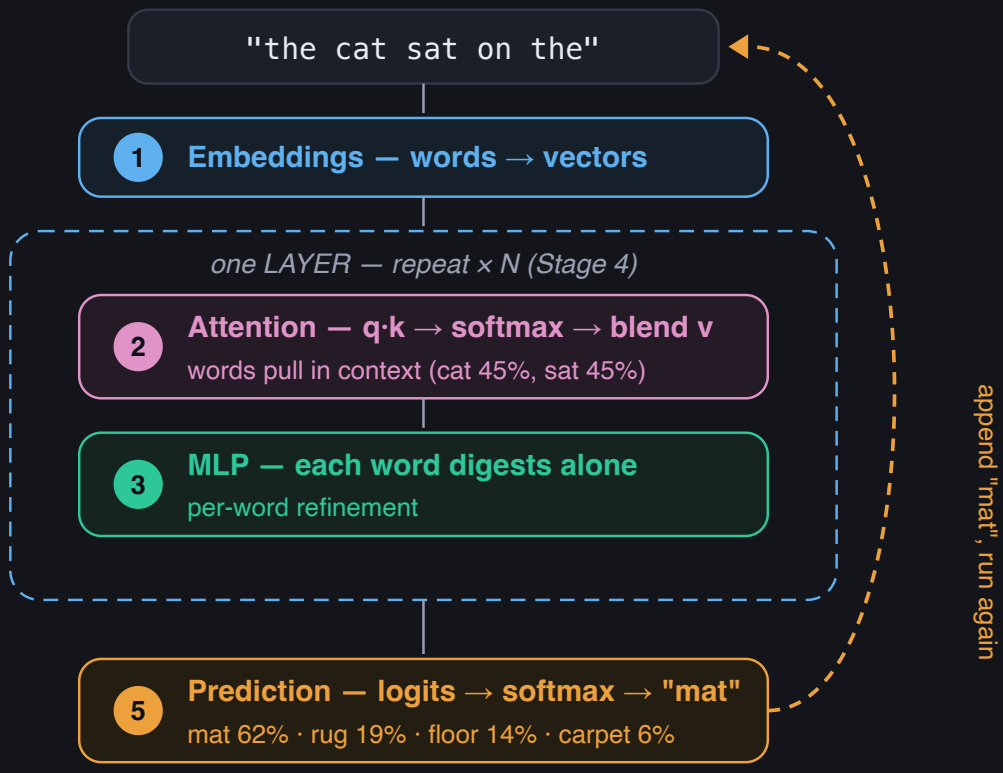


`"the cat sat on the"` → **mat** (62%). **You just predicted a word, by hand.**

Then the loop: append `"mat"` to the sentence and run the entire machine again to get the next word. And again. That predict→append→repeat loop *is* how an LLM writes whole paragraphs — one word at a time.

Checkpoint. Final vector → a logit per candidate word → softmax → pick one → append → repeat. ✓

The whole machine, in one view



The one-paragraph version

A transformer turns each word into a vector, then repeatedly lets the words **look at each other and exchange context** (attention) and **individually digest** it (the MLP). Stacking that gather-and-digest cycle builds a deep, context-aware understanding. It then turns the final vector into a **probability over the next word**, picks one, appends it, and repeats. The one idea that makes it all work is **attention** (Eq. (7)): score relevance with a dot product, softmax it into weights, and blend the values — which is exactly the arithmetic you just did for `"the cat sat on the → mat."`

What we kept out, on purpose: normalization (keeping numbers stable between layers) and tokenization (splitting text into sub-word pieces) are both real, but they're refinements bolted onto these five stages — not new ideas. Every modern architecture variant is a tweak to one of the boxes above.

Bridge to the modern transformer

Everything above is the timeless core. A *modern* model — like the one dissected in Aleksa Gordić's "The Life of a Token" — keeps all five stages but swaps fancier machinery into some of them. Here's the name-map (each name links to its source paper), so the jargon doesn't trip you up when you read it:

IN THIS DOC	NAME YOU'LL SEE	WHAT'S DIFFERENT
Positional encoding (add p_i, Eq. (2))	RoPE · YaRN	Instead of *adding* a position vector, modern models *rotate* each query and key by an angle that grows with position; YaRN stretches this to very long contexts.
MLP (Eq. (10))	SwiGLU · GeGLU	A *gated* two-branch MLP: one branch goes through the nonlinearity and multiplies ("gates") the other. Same role, a bit more expressive.
Normalization (the step we skipped)	RMSNorm · LayerNorm	Rescales each vector to keep numbers stable between layers. RMSNorm is the common modern choice.
Multi-head attention (Eq. (9))	MHA · GQA · MQA	The same heads — but several heads *share* one set of keys/values to save memory (GQA = grouped-query, the modern default).
Embeddings, attention core, residuals, prediction	(unchanged)	These are identical. The dot-product attention you computed by hand is exactly what runs inside every modern model.

That's the whole trick: if you can follow the five stages here, you can read his blog — you'll just be meeting new *names* for parts you already understand.

Where to go next

This document is the on-ramp. Here's a path onward, roughly easy → deep:

WATCH FIRST — VISUAL INTUITION

Attention in transformers, visually explained — 3Blue1Brown. The single best animation of the attention mechanism.

But what is a GPT? — 3Blue1Brown. The big picture, beautifully animated.

READ — BUILD THE FOUNDATION

The Illustrated Transformer — Jay Alammar. The classic diagram-driven walkthrough (original 2017 architecture).

The Life of a Token — Aleksa Gordić. The *modern* dense transformer in depth (RoPE/YaRN, RMSNorm, GQA) — the blog this doc bridges to.

DO IT — BY HAND AND IN CODE

AI by Hand — Prof. Tom Yeh. Worksheets that make you compute the matrices on paper.

Let's build GPT from scratch — Andrej Karpathy. Code a working transformer line by line.

The Annotated Transformer — Harvard NLP. The original paper as runnable PyTorch.

THE SOURCE PAPERS

Attention Is All You Need — Vaswani et al., 2017. The paper that started it.

RoFormer (RoPE) · **YaRN** · **GLU Variants (GeGLU/SwiGLU)** · **RMSNorm** · **GQA** — the modern pieces named in the bridge above.

External links were accurate at the time of writing; if one moves, a title search will find it.